

Pokémon The Series Search System

HaoChang Fu
up202108730@up.pt
Faculdade de Engenharia da
Universidade do Porto
Porto, Portugal

Afonso Vaz Osório
up202108700@up.pt
Faculdade de Engenharia da
Universidade do Porto
Porto, Portugal

Madalena Ye
up202108795@up.pt
Faculdade de Engenharia da
Universidade do Porto
Porto, Portugal

Abstract

*Pokémon*TM[3] the Series is an animation series that became popular after the release of the original *Pokémon*TM games, a staple of many people's childhoods. While some wikis store information about the franchise, they lack the flexibility to search by descriptive context rather than specific keywords, making it challenging for users to find memorable moments, episode numbers, or even characters based on descriptions.

With this in mind, this report outlines the process of designing a comprehensive search engine for episodes from *Pokémon*TM the Series. The project starts with gathering and processing data from Bulbapedia[2], which will then be thoroughly cleaned and structured for analysis. Using Apache Solr[20], the data is turned into a searchable collection. An exploration of Solr's features led us to a first iteration of a search system that showed good results compared to simple text processing, according to multiple metrics. Afterwards, we explored semantic search, which, when combined with the previous system, led to slight improvements. Lastly, we developed a Search User Interface that allows a user to interact with the system.

In the final system, the Mean Average Precision values are 0.64 for a complex lexical system, 0.38 for a purely semantic system, and 0.65 for a multi-stage system that reranks the results of the lexical system based on the complex system. When experimenting with queries closer to natural language, these values become 0.45, 0.40 and 0.46 respectively.

CCS Concepts

• Information Systems → Information Retrieval.

Keywords

Information Retrieval, Search, Search Engines, Pokémon, Data Scraping, Data Preparation, Data Processing, Pipeline, Document Indexing, Apache Solr

ACM Reference Format:

HaoChang Fu, Afonso Vaz Osório, and Madalena Ye. 2024. Pokémon The Series Search System. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (G66)*. ACM, New York, NY, USA, 19 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

G66, October 13–10, 2024, Porto, Portugal

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

This report is presented in the context of the **Information Processing and Retrieval** course, taught in the first year of the **Master in Informatics and Computing Engineering** in FEUP.

*Pokémon*TM the Series is a widely known animated series franchise that was inspired by the original *Pokémon* games. Its vast universe is enriched with memorable *Pokémon* and heart-warming storytelling which was passed down through generations of children. Many people were marked by its influence, but not many remember all the details from the *Pokémon*'s universe to proceed with the information retrieval task. As such, our goal is to develop an easy-to-access search engine with an intuitive interface that allows users to find information about their favourite animated series, episodes, characters or even specific *Pokémon* of their choice, by searching within the context of the storyline rather than relying solely on keywords. To achieve that, we needed to go through several phases to prepare the final product.

First of all, we need raw material to work on, said material in our case being *Pokémon* related data.

Data engineering is a complex task, requiring multiple steps in order to turn the raw data we collect into structured documents. In this paper, we will describe each of the steps needed to build up this information retrieval system. In **Data Sources**, we explain where we sourced our data. In **Data Collection** we describe how we collected said data, which is directly followed by **Data Preparation**, where we outline the transformations we applied to the data in order to improve its structure.

After a description of our **Conceptual Data Model**, we showcase some statistics we obtained based on our data and our interpretations of them in **Data Characterization**. Lastly, we present some **Prospective Search Tasks** and a brief summary of our **Data Pipeline**.

Moving on to the work of the second milestone, we describe our **Data Indexing** process, through which we turned our documents into a searchable collection. Afterwards, after defining our **Search Systems and Retrieval** process, we present the results of our **Evaluation** of the search systems, with regards to the information needs we defined.

In the final milestone, we explored the concept of **Semantic Search**. After reevaluating our systems taking into account the results obtained with semantic search, we developed another system that combines lexical and semantic approaches, using **Query Re-Ranking**. As an additional test, we explored how our systems behave when presented with more complex queries in **Information Need Searching**. We also tried out Solr's **More Like This** feature in order to provide more options to the user. Lastly, we present the **Search Engine and Front-end** we developed in order to have a user-friendly way to interact with the system.

2 Data Sources

Since there is a vast amount of trusted sources, we opted for an information source which is widely known by many *Pokémon* enthusiasts: the **Bulbapedia**[2]. Bulbapedia is a web page much like Wikipedia (in fact, it is build using the same tool, that being **MediaWiki**[12]), but centered in the *Pokémon* universe, which provides all the information in that context. It is a part of the Bulbagarden community website. It is also already a compilation of data related to *Pokémon* from other sources, such as the official *Pokémon* website.

The contents of Bulbapedia are available under the **CC BY-NC-SA 2.5**[10] license. This means we can freely use it for non-commercial purposes, as long as we provide credit and use the same license.

3 Data Collection

In order to extract structured information from the somewhat disorganized contents of web pages, it became essential to handle the *HTML* of the required pages. Conveniently, Bulbapedia, like all other MediaWiki pages, exposes a **JSON API** that gives us ease of access to its information, such as *HTML* code, by using keywords that identify a web page's address.

The API provided each web page as a *JSON* file that we could retrieve within our code. We used Python[17] as our main tool for data engineering to process this data. By accessing the *JSON*, we could retrieve the *HTML* field, allowing us to move on to the next step: extracting the relevant information. To accomplish this, we used a popular *Python* module frequently used for web scraping - **BeautifulSoup**[1]. Within the wiki, the relevant pages for our project were the following:

- **Characters:** the *HTML* displayed a table containing structured data related to a certain character (Figure 1) along with relevant sections of unstructured data, such as "Character"(describing personality traits) and "History" (providing a summary of the character's plot);
- **Episodes:** the *HTML* provided a table of structured data (Figure 2) and relevant sections of unstructured data such as "Plot", "Blurb";
- **Pokémon:** the *HTML* also showcased a table of structured data containing relevant information about the Pokémon (Figure 3), along with an additional set of unstructured data ("Biology", "Blurb");

Ash Ketchum
アッシュ・ケッチム

Ash Ketchum is featured in: <i>Pokémon: The Series</i>
Age: 10
Gender: Male
Eye color: Brown
Hair color: Brown
Height: 125 cm
Weight: 40 kg
Region: Kanto
Relatives: (None listed)
Relatives: Unnamed father (EP103)
Relatives: Unnamed grandfather (EP103)
Associated series: <i>Pokémon: The Series</i>

Figure 1: Character Table

Pokémon - I Choose You!
エピソード
ポケモン「I Choose You!
First broadcast
Japan: April 1, 1997
United States: September 8, 1998
English themes
Opening: <i>Pokémon Theme</i>
Japanese themes
Opening: <i>めざせポケモンマスター</i>
Ending: <i>ドラゴンボール</i>

Figure 2: Episode Table

Cyndaquil
ヒノアラシ
Fire Mouse
Pokémon
#0155

Images on the Bulbagarden Archives
Type: Fire
Abilities: Blaze, Flash Fire, Hidden Ability

Figure 3: Pokémon Table

The extracted data is initially stored in memory structures and subsequently saved to local storage as *JSON* files for processing in the next step.

Adding the size of all API responses we obtained, we parsed 368 MB of data for characters, 1386 MB for *Pokémon*, and 172 MB for episodes.

4 Data Preparation

Having collected the data, there's still a need for preparation and processing since there are a lot of values that could be easily analyzed after doing some operations over them. For this task, we used **Pandas**[16], a widely known data manipulation and analysis *Python* module. With *JSON* files loaded as **Data Frames**, we were able to identify a few issues:

- The data crawled had nested data structures (in the "Episodes" document) such as "First Broadcast", "Credits", "Characters" (characters that appear in an episode), "Japanese themes", "English themes", so we unfolded the children into separate columns to facilitate plotting. Some nested structures had to remain, notably arrays, such as the list of characters featured in a given episode;
- The date formats in the website were written in text, so we formatted them and standardized them as "mm/dd/yy" with numbers for every entry;
- The columns "Animated debut" and "Debut" were named inconsistently across character pages, so we merged them together as "Debut";
- There were some unnecessary columns (in the "Characters" documents) so we dropped them, such as "Game counterpart", "Manga counterpart(s)", "Manga series", "Games", "Generation" and "Counterpart(s)";
- The age attribute of the characters was not consistent and had letters mixed in, so we used regular expressions to extract numbers;

We also created some new columns, in order to provide some interesting information for later use in the data characterization step:

- Introduction of a new attribute "word_count" that counts the words in the plot for each episode;
- Introduction of a new attribute "broadcast_delay" that contains delay between the dates of broadcast in Japan and USA, in days;

5 Conceptual Data Model

After the previous processes, we can build up an idea of the main entities for our system which are the documents, that can be represented in this data model (see Figure 11):

5.1 The episodes entity:

- *epcode*: Unique identifier for each of the episodes;
- *name*: Name of the episode;
- *subtitle*: Subtitle of the episode;
- *plot*: The plot associated with each episode;
- *blurb*: A brief paragraph of the plot summary;

- *major_events*: A list of the major events that happened during the episode;
- *first_broadcast_japan*: The date that the first broadcast of the episode happened in Japan;
- *first_broadcast_USA*: The same as the previous item but in the US;
- *broadcast_delays*: The delay between the broadcast dates in Japan and the US;
- *english_opening_theme*, *english_ending_theme*, *japanese_opening_theme*, *japanese_ending_theme*: Themes for the animation;

5.2 The characters entity:

- *character_code*: Unique identifier for each of the characters;
- *name*: Character name;
- *age*: Character age;
- *character*: Character trait description;
- *gender*: Character gender;
- *eye_color*: Character eye colors (might have multiple due to many variants);
- *hair_color*: Character hair colors (might have multiple due to many variants);
- *role*: A character's main role in the series;
- *first_appearance*: The series/game/movie that a character appeared the first time in;
- *history*: Brief description of a character's background in the series;
- *relatives*: The relatives of a character;
- *debut*: The series that a character debuted in;
- *hometown*: The hometown of a character;
- *region*: The region that a character is from;
- *animated_series*: The series that a character participated in;
- *trainer_class*: The jobs/trainer classes a character has/had;
- *member_of*: The organization that a character is part of;
- *rank*: The rank of a character as a *Pokémon* trainer;

5.3 The *Pokémon* entity:

- *pokedexEntry*: The unique ID for a *Pokémon*;
- *name*: The name of a *Pokémon*;
- *abilities*: the set of abilities a *Pokémon* has;
- *image*: The image URL associated with a *Pokémon*;
- *biography*: A description of a *Pokémon*, it's background and it's behaviour in general;

6 Data Characterization

After processing the documents, we analyzed it with multiple measures in order to characterize our collection. The plots shown were made using the **Matplotlib**[11], **WordCloud**[28] and **NLTK**[15] Python modules. Overall, our dataset has 247 entries for episodes, encompassing the entirety of the original series. Down the line, we will add the episodes from later seasons, all of which have pages in Bulbapedia with the same format we processed previously.

We computed some statistics regarding the characterization of the episodes, as shown below.

Table 1: Episode Characterization

Field	Plot word count	Nº of Pokémons	Nº of Characters	Broadcast delay
Max	2850	142	43	1061.0
Avg	1077	20	11	214.7
Min	373	4	2	2.0

6.1 Episode Word Count

As we can see in the histogram presented in figure 4, word count is fairly consistent across the different plots, most of them being around 750 to 1000 words. The shorter episodes are around 500 words, and only a few go beyond 1500.

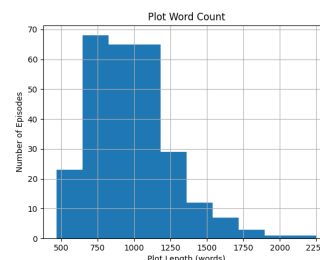


Figure 4: Word count graph

6.2 Episode Plot Word Cloud

The word cloud shown in figure 5 gives us an overall look at what the plots, our main source of unstructured text, contain. As expected from the plot of television episodes, character names and show-specific terms are abundant, such as "Ash", "Misty", and "*Pokémon*". Verbs are also very frequent, which is also unsurprising, given that a plot can be summed up as a description of a series of actions. Having a closer look, we can also catch a glimpse of some of the show's themes of friendship, such as "friend" and "everyone", as well as of *Pokémon* battles, with "attack", "battle" and a series of *Pokémon* moves.



Figure 5: Word Cloud

6.3 Correlation Between Characters

We also envisioned a metric that would allow us to express how close two characters are in terms of their appearances, by dividing the number of episodes in which they both appear by the number of episodes in which either of them appears, which we later learned corresponds to an existing measure called a Jaccard index. This allows us to infer relationships between characters (for example, the character Dr. Namba, a Team Rocket scientist, is correlated with other Team Rocket characters). However, this metric can be misleading for characters without many appearances, as two characters who only show up once and in the same episode would have a perfect correlation, making it more "reliable" for characters with more occurrences.

6.4 Representation of *Pokémon* Types

We compared the prevalence of the different *Pokémon* types by plotting the occurrences of types on all Generation I and II *Pokémon* (corresponding to the seasons of the show we collected) together with the occurrences of *Pokémon* of said type in individual episodes, normalized to a scale of 0 to 1, as can be seen on figure 6. In both cases, Water-type *Pokémon* are the most prevalent. As for other types, there are cases of under and over-representation in the episodes. Some cases of over-representation can be easily explained by looking at the *Pokémon* of the main characters: for example, Ash has a Pikachu, and Team Rocket makes frequent use of Poison-type *Pokémon*.

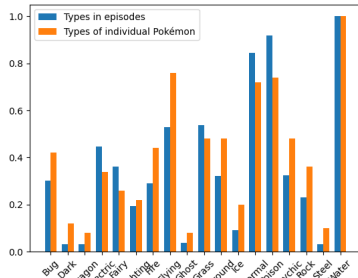


Figure 6: *Pokémon* Types

6.5 Broadcast delay between Japan and the US

When the original show aired, the market for dubbing, or producing an English localization of, Japanese animated series wasn't as big as it's today, and took much longer, which is why it's interesting to look at the delay between the original and US broadcasts (in days). Missing values are due to there simply not existing episodes airing at the time. We can notice a general trend: as the series picked up steam outside Japan, the delay shortened and became somewhat consistent. This figure (fig. 7) also allows us to quickly spot outliers: for example, episode 18 took over three years to air in the US due to the episode having multiple controversial scenes that had to be edited or outright cut[6]. The period during which there were no episodes was due to the series going into a hiatus after a controversial episode, which threw the production into disarray[7], hence the inconsistency shortly after the series resumed airing.

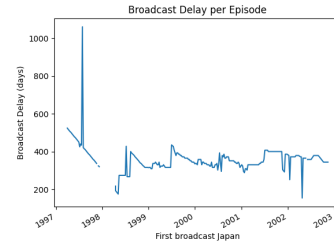


Figure 7: Broadcast Delay

6.6 Number of major events per episode

Each episode is accompanied by a list of "Major events", and we plotted their distribution (fig. 8). We can see that the majority of episodes have only one event, while a few have many more, indicating that they are more relevant to the overarching narrative of the series. However, as we can see in Figure 9, the amount of major events is not strongly correlated to word count.

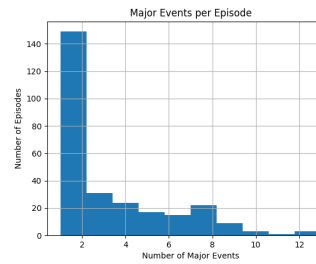


Figure 8: Distribution of number of major events

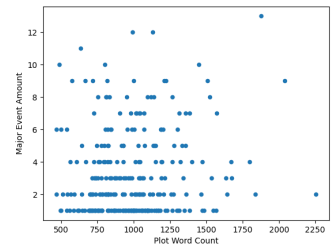


Figure 9: Number of major events against word count

6.7 Sentiment Analysis on each Episode

We conducted a sentiment analysis of the plot of each episode as described by the figure (fig. 13). The results show that most episodes are categorized as negative, although there are still many positive ones. This is mainly because, despite the positive outcomes and moral lessons of each episode, the journey to reach those conclusions often involves negative elements, such as frequent battles or the recurring appearance of Team Rocket attempting to steal Pokémon.

7 Prospective Search Tasks

Through our characterization of the documents, we gained a better understanding of the kind of information that could be gleaned from them.

Thinking back to the most common kind of terms found in the word cloud of the plots, the characters and their interactions are a big focus. Queries such as "**episode with James and Professor Oak**" or "**episode in which Ash battles Jessie**" could be used to find episodes according to what occurs during them, giving extra weight to those classified as "Major events" in the document. This sort of query can be extended to other events, such as a given

Pokémon being caught. This is useful in the case of someone wanting to pinpoint an episode they remember only part of.

By integrating the information from the character and *Pokémon* pages into the documents, broader and more meaningful queries can be made, such as asking for "**episodes in which Ash battles a Gym leader**", or "**episodes in which a Water-type Pokémon is caught**".

For those more interested in the overarching narrative, "Major events" can be used as a filter in order to ignore "filler" episodes. Users could ask, for example, for "**most important episodes for Misty**", or "**most consequential episodes**".

8 Data Pipeline

With all that said, we can picture a pipeline where the input is the raw data materials coming from the web and the output is the organized information for our system. Each step brings the collected data closer to the desired result, while providing us statistics that help us in understanding the characteristics of the collected data, in order to grasp what informational gains potential users can glean from it. The result, a diagram presenting the actions and entities involved in our pipeline, according to what has been described in previous sections, is presented in Figure 12.

9 Data Indexing

Having prepared our collection of documents, we then further processed it for indexing in a search tool that allows us to define queries that satisfy our information needs. The tool we used is **Apache Solr**[20]. We had to make some modifications to our scripts, in order to present the collection in a single file, in the format that Solr expects. This section describes the final structure of our documents and the steps we took in order to index it into Solr.

9.1 Document Definition

The document structure we settled on before indexing is the result of a combination of the collections we obtained in the first milestone. Episodes of the series are the root documents. During the indexing process, we preserved most of the data collected for these documents.

Using Solr's native support for nested documents, we merged some data from the character and *Pokémon* collections into the episodes. We excluded multiple columns from these documents, because we felt they weren't relevant for our goals with this search system, and/or because they only applied to a handful of characters, with inconsistent meanings. The structure of our documents in this merged collection will be outlined after our data types are presented.

9.2 Indexing Process

Indexing is a relevant process in Information Retrieval, as it allows searching to be more efficient and take less time, by organizing the data.

In Solr, there are several ways to index document fields and related queries, namely Tokenizers [22] and Filters [21]. The first one generates a sequence of tokens from a starting string, following a set of rules; and the second one transforms these tokens so that they can be matched more easily later.

In our project we used:

- **StandardTokenizerFactory** - a tokenizer that splits the text upon whitespace or punctuation;
- **EdgeNGramTokenizerFactory** - a tokenizer that forms n-grams starting at the edge (the first character) of the string;
- **EdgeNgramFilterFactory** - same as above, but as a filter;
- **ASCIIFoldingFilterFactory** - a filter that replaces special characters with their ASCII counterpart;
- **LowerCaseFilterFactory** - a filter that replaces uppercase characters with their lowercase alternative;
- **PorterStemFilterFactory** - a filter that applies stemming to all the words, converting them to their roots;
- **BeiderMorseFilterFactory** - a filter that allows the system to identify similar names or words by applying the Beider-Morse Phonetic Matching algorithm;
- **SynonymGraphFilterFactory** - a filter that expands words to include their synonyms, based on a synonyms file provided to the system;
- **FlattenGraphFilterFactory** - a filter that must be used after graph-based filters.

Combining these filters and tokenizers, we used different field types that serve different purposes, mostly by changing the **index** and **query analyzers** of text-based fields:

- **string**: Standard string type that belongs to the **Solr.StrField** class;
- **pint**: Default int field type that belongs to the **Solr.IntPointField** class;
- **pdate**: Default field type for date-time formatted as "YYYY-MM-DDThh:mm:ssZ", using a subset of the ISO-8601[8] format;
- **textNoSynonyms**: This field type belongs to the **Solr.TextField** class and uses **StandardTokenizerFactory**, **ASCIIFoldingFilterFactory**, **LowerCaseFilterFactory**, **PorterStemFilterFactory**. After transforming the text into a simpler format (lower case, ASCII), we use stemming so that we can match different variations of the same term;
- **text**: This field is identical to the previous one, but uses **SynonymGraphFilterFactory** and **FlattenGraphFilterFactory** filters. With this, we can define a list of synonyms for the words in the text field so we can get more varied matches without having to specify multiple terms with the same meaning.;
- **pokemonText**: This field is identical to **text**, but the list of synonyms used is crafted specifically for the Pokémons. Since we want to use this type for the multivalued field **major_events**, we have to set a high **positionIncrementGap** to avoid spurious phrase matches between terms in different items. We used a value of 100;
- **auto-complete**: This field type uses a different approach, to allow it to be used later on as an auto-complete for the user. It uses the same tokenizer and filters as **textNoSynonyms**, with an added **EdgeNGramFilterFactory**, to allow us matching with the n-grams of a words during the query (to simulate user typing letter by letter of a specific word);
- **name**: This field type is made specially for the fields containing names, it allows a user to search a name by similar

sounding word during the querying process by using **BeiderMorseFilterFactory**, asides from the **LowerCaseFilterFactory** and **ASCIIFoldingFilterFactory** to standardize the text;

- **epcode**: Also made for the purpose of auto-completion, this field uses a **EdgeNGramTokenizerFactory** to allow users to match episodes by the start of their code, in which the first two characters identify the season (e.g. "XY" of "XY014"), and/or the subsequent episode number (eg. "XY01" of "XY014").

Nearly all field types have the same filters and tokenizers in their **query analyser** and **index analyser**. The exception is **text** and **pokemonText**, because of synonyms. Since words are already expanded to their synonyms in the index, we don't need to do it again at query time, since one synonym is all we need to get a match. This is the recommended way of handling synonyms in Solr[27], even if processing them at query time leads to a smaller index.

When using generic synonyms across the whole system, we got unexpected results that harmed the performance of the system. Thus, we built our own **synonyms.txt** file, by combining all synonyms for the adjectives present in the Pokémon's "Biology" field, so that we can more easily match Pokémon with a given personality. To obtain these synonyms, we used the NLTK Python module[15]. The **pokemonText** type uses a different file, **pokemon_synonyms.txt**. This one was created by going through each Pokémon and adding a line of the form "<pkmn_name> => <pkmn_name>, pokemon". This approach makes it so that the term "Pokémon" matches any Pokémon name, but not vice-versa.

Our indexed fields are as follows:

- **episodes**: title (auto-complete), plot (pokemonText), major_events (pokemonText), first_broadcast_japan (pdate), first_broadcast_united_states (pdate), epcode (epcode), title_query (textNoSynonyms);
- **pokemon**: name (name), blurb (text), biology (text)
- **character**: name(name)

Each document type also contains multiple non-indexed stored fields, that can provide additional information to the user when retrieved. **title_query** is a non-stored field used so that we can query over the titles of the episodes. This isn't doable with the main **title** field since we left that field for future use in auto-completion.

10 Search Systems and Retrieval

We developed two schemas, one more complex than the other, in order to compare their performance and assess the quality of the complex schema.

The simple schema uses only the default types provided by Solr, and a simple text-based field with ASCII folding and lowercase filtering.

The complex schema takes advantage of more of Solr's features by using the custom field types described in the previous section.

To retrieve only episodes, we have to use the **fq** (filter query) parameter in order to exclude children. We achieve this by using the value `-_nest_path_*`, in which `_nest_path_` is a field created automatically by Solr which describes how the document is nested. Root documents do not have this field.

For our queries, we used Solr's eDisMax query parser[5], which allows queries to be defined in plain text, and offers multiple parameters that we can use to improve the results provided by the complex system. Under the hood, this parser creates multiple sub-queries and combines their results in order to produce a ranked list of results. We used three of the query parser's parameters:

- **qf (query fields)**: Defines which fields are to be used in the search. Different boosts can be applied to each field.
- **pf (phrase fields)**: Defines which fields are to be considered when searching for phrase matches. Boosts can also be applied here.
- **ps (phrase slop)**: Used to determine how close the searched terms have to be in order for them to be considered a phrase in the fields defined in **pf**.

Table 2: eDisMax parameters

	Value
qf	title_query^7 major_events^10 plot^4
pf	plot^25 major_events^25
ps	6

We assigned a large boost to the **major_events** field, since it usually describes events of critical importance to the story. The **title_query** field also gets a boost, but it is rarely matched when searching for the content of an episode, since the titles in this series aren't very informative with regards to what occurs during it. However, it can be useful at times. As for phrase matching, we assigned equal boosts to the **plot** and the **major_events**, with a phrase slop of 6. This gives a much higher score to results in which all the searched terms occur in close proximity.

In the simple system, we used the same query parser with the same values on these fields, with the boosts removed. This allows us to examine the effect that these boosts, along with the additional processing done during the index process, have on the performance of the system.

Outside the eDisMax parameters, we used the standard Solr parameters, such as the **q** (query) field and the **q.op** (query operator) field, for which we used **AND**.

11 Evaluation

Now that we have defined our retrieval process, we must assess the performance of our search system with regards to the information needs we define for it. Our goal is to determine the capacity of the system to return relevant documents in a ranked list of results, with no concern of how relevant they are relative to each other or of how efficient the retrieval process is.

To get clear results, we need metrics that can give us insights into the performance of the search system given two inputs: a ranked list of results retrieved from the system, and a set of relevant documents, evaluated manually by us. To this end, we can use multiple metrics derived from precision and recall. Our collection is composed of 1077 episodes (without taking into account nested documents), so manually determining each episode's relevance would be extremely inefficient and time-consuming. To alleviate this, we used a subset

of documents for each query, corresponding to the union of the top 30 results returned by both systems (simple schema and complex schema).

The **Average Precision (AvP)** allows us to condense the quality of the search system across recall levels into a single query. It is commonly used for comparing systems.

The **Mean Average Precision (MAP)** is among the most commonly used measures in Information Retrieval. It is the mean over the AvP values for a given set of queries, and is used for overall comparisons of different systems.

Precision at K (P@k) is a metric that focuses only on the precision of the top k results. In many contexts, such as web search, the user will want relevant results at the top of the list, and probably won't examine the rest. In other words, high recall may not be an important goal for the system.

Lastly, for a visual representation of the system's performance for a given information need, we can plot out **Precision-Recall curves**. With these curves, we can see how effectively the system is able to maintain a good precision as the recall increases. These curves also provide us with another metric, that being its Area under Curve (AUC), which lies between 0 and 1 (ideal system).

Taking into account the kind of information that can be gleaned from an episode's plot, we devised multiple information needs that could be used to evaluate our systems. For each query presented, the Precision-Recall curves for both systems can be found in the Appendix.

11.1 Recurring Themes

First, we tried to retrieve episodes based on its thematic elements. Stealing Pokémon is a recurring theme in the show, usually involving the Team Rocket Trio. As such, our information need is finding **episodes that involve Pokémon theft**. Our query is **steal pokemon**.

Table 3: Pokémon theft results

Metric	Simple	Complex
AvP	0.39	0.66
P@10	0.7	0.9
P@20	0.7	0.85

Both systems provide relevant results, despite the low AvP values: as we can see, they maintain similar precision after 20 results. This is due to each system retrieving different relevant documents, "diluting" the pool of documents, which is also visible in the P-R curves (Figures 14 and 15) as a sudden drop in precision. This is to be expected in this case, since stealing Pokémon is something that happens very frequently across the entire show. Still, the complex system performs better. Taking a closer look at the documents misidentified by the simple system, we can see that it returns multiple documents that, while involving theft and Pokémon, do not involve the theft of Pokémon in particular. The phrase slop, combined with the Pokémon synonyms, helped the complex system overcome this problem.

11.2 Major Events

Over the years, Ash's group has shifted time and time again. A big deal is usually made out of these changes in the group, enough for it to be consistently considered a "Major Event" according to Bulbapedia. Our information need here is finding **episodes in which Ash's group gains or loses a member**. We are only considering human characters, with one exception: we considered Team Rocket's Meowth to be close enough to a human, since he can talk. Our query is **(leave OR join) group**.

Table 4: Ash's group results

Metric	Simple	Complex
AvP	0.11	0.91
P@10	0.2	1
P@20	0.25	0.9

In this case, the simple system performed very poorly compared to the complex one. This is mostly due to the lack of stemming, as different conjugations of "leaving" and "joining" aren't taken into account.

Another important information need regarding major events and the overall narrative is to find **episodes in which Ash obtains a gym badge by winning a battle or through other means**, because a user may be interested in how many badges the main protagonist has or finding specific moments of Ash battling a gym leader. This excludes other institutions or events such as tournaments because we want moments related explicitly to the gyms. Since winning gym battles usually leads to earning badges and they are closely related, we considered both situations relevant: for example, even if Ash didn't win a badge by winning against a gym leader but helped a gym leader in some way (for example, by defending the gym or its Pokémon), it would count as Ash interacting with the gym leader and earning the badge, therefore as a win. Our query is **ash wins gym battles**.

Table 5: Ash winning gym battles results

Metric	Simple	Complex
AvP	0.28	0.89
P@10	0.6	0.9
P@20	0.55	0.9

In this case, the second system performed considerably better. Despite not having a terrible P@10, the simple system retrieves multiple irrelevant documents due to multiple factors. Firstly, the lack of phrase slop means it retrieves documents in which, for example, "winning" isn't related to Ash's outcome in the battle. This is exacerbated by the fact that this query has more terms (4) than the other ones presented, which means there's more room for matching unrelated occurrences of queried terms. The lack of a boost in the **major_events** field was also a detriment, since these sorts of victories are usually highlighted there in a succinct manner. We also noticed that, more so in this query than for the other ones,

the boost in the **title_query** field was useful, since the fact that the episode involves a gym tends to be mentioned in it. Usually, the titles are little more than puns or catchy sentences.

This case also shows us that our query doesn't need to be a direct summary of our information need, as gym battles and badges are deeply connected in the context of the show.

11.3 Show-specific Terms

With each generation of games, a new gimmick is introduced, and said gimmick is usually represented in the show. For the sixth Pokémon generation, it was the concept of "Mega Evolutions". Our information need is finding **episodes in which a Pokémon Mega Evolves**. Our query is **mega evolve**.

Table 6: Mega Evolution results

Metric	Simple	Complex
AvP	0.44	0.87
P@10	0.9	0.9
P@20	0.7	0.9

For this case, both systems perform similarly, until a certain recall threshold. Firstly, unlike with the Pokémon theft query, since Mega Evolutions occur mostly within a single season, there is more overlap in the relevant documents retrieved by both systems, avoiding the drop in precision that was seen in that case. However, once again, the simple system fails to retrieve many relevant documents due to a lack of stemming, as the term appears in multiple forms throughout the episodes (Mega Evolved, Mega Evolving, ...). Still, as the P@10 and P@20 values show, it still retrieves relevant documents. The lack of phrase slop also led it to retrieve documents from earlier seasons in which moves with names such as "Mega Punch" were used. Lastly, some documents were retrieved where, despite the plot mentioning Mega Evolutions, no actual evolution took place during the episode. This last problem applies to both systems.

Taking into account these four queries, the **Mean Average Precision** for the simple system was 0.31, while for the complex system, it was 0.83. As expected, the filters and field boosts added in the complex system yield better results than using basic text processing.

Table 7: Mean Average Precision

System	MAP
Simple	0.31
Complex	0.83

12 Evaluation on nested documents

Since our collection includes nested documents, describing the Pokémon and human characters present in each episode, we also prepared a query to evaluate how our systems perform when retrieving them. We reached the conclusion that the human character

documents aren't very interesting when relating them to their parent episodes, since their descriptions tend to mention aspects of the character's history. Thus, when we want to search, for example, for episodes with characters belonging to Team Galactic, any character who is tangentially related to Team Galactic will produce a match. The Pokémon, however, have more potential to enrich our system.

The **synonyms.txt** file we created for the Pokémon was based on adjectives so that we could find Pokémon based on their personality. One information needs that fits that category is finding **Pokémon that tend to be angry**. While our final goal is to use the results of these needs as "filters" to retrieve episodes, we chose to evaluate this query without joining the Pokémon to their parent documents. This can be easily added to existing queries using the Block Join Parent Query Parser[24]. Since each Pokémon tends to have multiple appearances on the show, having many episodes matching for a single Pokémon that was correctly identified would obfuscate the effectiveness of the child document search.

The query used for this information need was simply **angry**, and the parameters were as follows:

Table 8: eDisMax parameters for Pokémon

Value
qf biology^10 blurb^2

Table 9: Angry Pokémon results

Metric	Simple	Complex
AvP	0.14	0.59
P@10	0.4	0.7
P@20	0.35	0.6

For this case, the simple system fails to retrieve even 30 documents, since it requires the word to be specifically "angry". The complex system, despite retrieving some irrelevant results, has a much broader reach, thanks to the use of synonyms.

13 Additional Signals

For a brief period of time, we experimented with adding additional signals from our collection into the score as independent boosts. For example, we experimented boosting the scores of episodes based on the amount of major events it contains, using a logarithmic scale. However, this proved to be highly situational, since, for many queries, the amount of major events is irrelevant. We did not explore this idea in more depth.

14 Semantic Search

While the methods we have implemented so far have proved to be efficient for the information needs we defined, a purely lexical approach to information retrieval has its limitations, as it is unable to extract meaningful information on the semantic relationship between the terms and on how those relationships affect what the written text is expressing.

To integrate semantic search into our system, we used **dense vectors**. In this context, a dense vector (or embedding) is a data structure that allows us to encapsulate the semantic meaning of a document’s textual fields in a fixed number of dimensions.

This functionality was added with the latest major release of Solr (version 9). As such, while functional, it is lacking in options, which limited what we could do in order to tailor them to our use case. This limitation stemmed mainly from the fact that the current versions of the dense vector query parsers do not allow searching over multi-valued dense vectors. However, this feature is being worked on, as we can see in Apache Lucene’s GitHub repository[14]. This is a particular problem for us, as we have multiple text fields per episode, one of which is multi-valued itself (**major_events**). Another possible approach would be to consider each embedding as a child document of the episode, and use the **Block Join Parent Query Parser** to combine their results. Although this is fairly cumbersome, and leads to a significant increase in document number, this allows us to separate the embeddings into chunks that, semantically, make sense (paragraphs for the plot, bullet points in a list for the major events).

14.1 Choice of Model

To choose a model to create our embeddings, we explored the HuggingFace leaderboard for pre-trained models[9]. Among the models that scored higher for semantic search, we ended up using **all-mpnet-base-v2**, due to its maximum sequence length and high amount of training data. Since this model’s maximum sequence length is 512, and, as we observed previously in Figure 4, the word count for a given episode’s plot rarely goes beyond 1500 words, since the text is split into paragraphs, it is highly unlikely that the model will truncate any input we give it.

14.2 Indexing Dense Vectors

After obtaining our embeddings through a Python script, using the **sentence-transformers**[19] module, we added a new field “vector” to our schema, which is only present in the child documents containing an embedding. This field uses the **Solr.DenseVectorField** class, and uses the dimension and proximity function corresponding to our model: 768 dimensions, and dot product similarity.

14.3 Query Parameters

When using queries based solely on the embeddings, we used the **knn Query Parser**[23], searching for the 500 nearest neighbors. We used this large number due to each episode having multiple embeddings: if all embeddings of multiple episodes are present at the top of the list, it may cause other episode’s results to be ignored. Moreover, we noticed that increasing this number didn’t seem to have a noticeable impact in query response time for our collection. Since the embeddings are child documents, we also used the **Block Join Parent Query Parser** to aggregate the results. Initially, we used the **total** scoring method, which sums the scores of all matching children. However, when observing the results, we noticed that this was unfairly giving high scores to episodes whose plots are split into a high amount of small paragraphs. Therefore, we switched to the **max** scoring method.

15 Semantic Search Evaluation

When updating our query relevance judgements to take into account the results returned by the semantic system, we found new relevant documents. As such, since there are now more known relevant documents, it is expected that the AvP values overall will be lower when reevaluating the complex lexical system. For this evaluation, we split the information needs we presented previously into two categories, taking into account whether they rely on domain-specific knowledge or not.

15.1 Generic Needs

For this group, we used the **steal pokemon** and the **(leave OR join) group** queries. While **pokemon** is a domain-specific term, we considered that the act of stealing something is generic.

Table 10: Pokémon theft semantic results

Metric	Complex	Semantic
AvP	0.29	0.28
P@10	0.9	0.9
P@20	0.85	0.9

Table 11: Ash’s group semantic results

Metric	Complex	Semantic
AvP	0.66	0.61
P@10	1	1
P@20	0.9	0.85

For this group of information needs, the semantic system performs very similarly to the complex lexical system. Despite having slightly lower **AvP** values, the **P@10** remains the same, meaning the relevance of results that users are the most likely to look at is unchanged.

15.2 Domain-Specific Needs

For this group, we used the remaining two queries: **mega evolve** and **ash wins gym battles**. While the second query doesn’t contain any Pokémon-specific terms, a “gym” in Pokémon has a very specific meaning, which separates it from a real-world gym.

Table 12: Ash winning gym battles semantic results

Metric	Complex	Semantic
AvP	0.78	0.13
P@10	0.9	0.3
P@20	0.9	0.35

Table 13: Mega Evolution semantic results

Metric	Complex	Semantic
AvP	0.81	0.49
P@10	0.9	0.9
P@20	0.9	0.75

For these queries, the semantic search system performed noticeably worse than the previous system. This is most visible in the gym query, in which it performs even worse than the simple lexical system. However, for the Mega Evolution query, while the **AvP** value is low, the **P@10** and **P@20** do not change much, which means that the model was still able to somewhat capture the meaning of the query (the non-relevant results are more prevalent after rank 20).

Table 14: Semantic Mean Average Precision

System	MAP
Complex	0.64
Semantic	0.38

16 Query Re-Ranking

Despite the poor performance of the semantic search in some cases, it still has potential. By doing little more than applying a pre-trained model to our collection, we were able to achieve somewhat similar results when compared to a system that required manual effort in figuring out how to configure the eDisMax weights, which filters to use, and in building synonyms. Therefore, we used Solr’s **Re-rank Query Parser**[25] in order to perform a two-stage retrieval: our complex lexical model is used to retrieve the top results, which are then reordered with the semantic system.

Our hypothesis with this system is that, by limiting the scope of the semantic search through an initial filtering with the lexical system, it will be able to give priority to the documents that are relevant, while avoiding the senseless matches we observed when it was done over the entire collection. We used a **reRankWeight** of 4, meaning that the result of the re-rank query will be multiplied by 4 and added to the initial score. Taking into account the score values we observed, this value gives the re-rank some weight while not allowing it to completely overtake the initial score.

Table 15: rerank parameters

	Value
reRankWeight	4
reRankDocs	30

16.1 Query Parameters

For this system, we used exactly the same parameters as for the complex lexical system, with the addition of the **rq** parameter, which lets us define a second query that will be used to rerank the initial results. The rerank query uses the same parameters as the queries used in the purely semantic system.

16.2 Generic Needs

Table 16: Pokémon theft rerank results

Metric	Complex	Semantic	Rerank
AvP	0.29	0.28	0.29
P@10	0.9	0.9	0.9
P@20	0.85	0.9	0.9

Table 17: Ash’s group rerank results

Metric	Complex	Semantic	Rerank
AvP	0.66	0.61	0.68
P@10	1	1	1
P@20	0.9	0.85	0.95

For this group, the rerank system performs slightly better than the complex system. By taking a closer look at the **P-R curves** (figures 24 to 29), we can see that the irrelevant results were mostly moved down in the ranking, with the exception of one result in the theft case. Still, in both cases, the use of re-ranking improved the **P@20** value.

16.3 Domain-Specific Needs

Table 18: Ash winning gym battles rerank results

Metric	Complex	Semantic	Rerank
AvP	0.78	0.13	0.80
P@10	0.9	0.3	1
P@20	0.9	0.35	0.9

Table 19: Mega Evolution rerank results

Metric	Complex	Semantic	Rerank
AvP	0.81	0.49	0.82
P@10	0.9	0.9	0.9
P@20	0.9	0.75	0.9

Here, we can see the same behavior as in the generic needs. Irrelevant results are generally moved down in the ranking, leading to improved (or equal) **P@10** and **P@20** values.

17 Information Need Searching

In order to test our systems with input that would be more "natural" to a user, we used two queries, based on our previous information needs, that are closer to a written sentence. The **ash wins gym battles** wasn't included since it is already a sentence, and **mega evolve** wasn't included since it amounts to little more than a keyword search. As such, **steal pokemon** became **someone steals pokemon**, and **(leave OR join) group** became **someone leaves or joins the group**.

Table 20: Pokémon theft sentence results

Metric	Complex	Semantic	Rerank
AvP	0.18	0.23	0.15
P@10	0.6	0.8	0.4
P@20	0.65	0.8	0.55

Table 21: Ash's group sentence results

Metric	Complex	Semantic	Rerank
AvP	0	0.67	0
P@10	0	1	0
P@20	0	0.95	0

As we can see, these more elaborate queries have a massive impact on the performance of the lexical system (and, as a consequence, the rerank system). For the second case, the complex lexical system was only able to retrieve three results, none of which were relevant. The purely semantic system, however, is barely affected by these changes, even providing better results for the second case. Since the underlying models are trained on natural language, adding "filler" words that don't change the semantic meaning of the query will have little effect on the generated embeddings.

To avoid this problem in the future, the practice of query processing could be applied, in order to break down complex queries into its key terms, resulting in queries such as the ones we used throughout the project.

Table 22: Final Mean Average Precision

System	MAP	MAP*
Complex	0.64	0.45
Semantic	0.38	0.40
Rerank	0.65	0.46

Although the similarity of **MAP** values between the complex lexical system and the rerank system is to be expected, since they only differ in ranking, we were able to show that the reranking process is able to sufficiently increase the scores of relevant results, leaving the irrelevant ones further down the list. In the **MAP*** column, which also takes into account the sentence queries from section 17, we can see that the value for the lexical (and thus rerank)

system falls considerably, bringing it closer to the slightly increased value of the semantic system.

18 More Like This

Apache Solr[20] provides a helpful feature called More Like This[13] that is used to find documents that are similar to a given document. We experimented with this feature in order to help users who want to discover episodes with similar themes, characters, or storylines.

There are several ways to use More Like This. The recommended one is to use it as an additional request handler, which we did. For our case, we used a script to precompute and store recommendations for similar episodes in a JSON file. The script systematically iterates over all Pokémon episodes, querying Solr's MLT feature for each episode to find other 5 episodes with high similarity scores. The parameters used in the queries were as follows:

- **"mlt.fl = plot, title, major_events"**: it specifies the fields used for similarity
- **"mlt.mintf = 2"**: minimum frequency below which terms will be ignored in the source document.
- **"mlt.mindf = 6"**: minimum frequency below which terms will be ignored that do not occur in at least this many documents.

Internally, term vectors are generated, and the results are scored with regards to the overlap between the terms in the queried document and the rest of the collection.

18.1 Evaluation

Evaluating this feature required a different approach compared to previous methods for assessing information needs. Comparing the semantics and similarity between two texts, such as the plots of Pokémon episodes, is inherently complex and often subjective, especially when done manually. To tackle this, we used Python's SpaCy[26] library to bring an objective perspective to the evaluation process.

SpaCy is a powerful tool for advanced natural language processing (NLP). It allows us to compare and measure the similarity between texts by analysing their contextual relationships. By using SpaCy, we were able to systematically and objectively assess how well the plots of episodes aligned with their recommended similar ones. This helped ensure that the recommendations made sense from a semantic and narrative perspective.

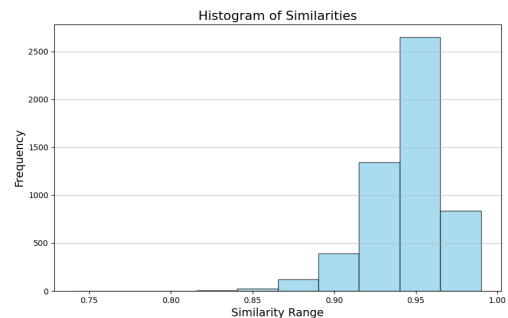


Figure 10: MLT Similarity Evaluation

We saved the results (the degrees of similarity) in a new JSON file and created a histogram (figure 10) to visualize them and how often they occur.

The overall average similarity between the episodes is quite high. Most similarity scores in the histogram are clustered around 0.95. While this may suggest that the MLT request consistently retrieved episodes that are in fact very similar, this may be somewhat biased due to the context of our collection. In most long-running shows like Pokémon, there will be a lot of tropes and formulaic elements, naturally leading to a high similarity among episodes. Still, this is a useful feature that users will be able to take advantage of the Search User Interface in order to find episodes with shared themes or interactions.

19 Search engine and Front-end

To represent the search system in a user-friendly interface, we created a website built on **React**[18] for client-side UI rendering and **Django**[4] for back-end processing. The back-end serves as a middleman to compute embeddings for the user's query and to redirect the requests to the Solr system with predefined query parameters, so the user only has to provide the query/question text (as seen in the annexes). We provided two types of search for the user, each with its corresponding endpoint in our back-end, that can be changed through a single switch:

- Search for the *Pokémon*: this allows users to search information about specific *Pokémon* (complex lexical system);
- Search for the episodes: this allows users to search for the pretended animated series episodes (re-rank system);

The website provides a list of results that can be filtered according to some criteria and provides something similar to pagination, which is limiting the number of entries a user can see, since our approach to a result page was done in a list of scroll items (shown in this picture). For both Pokémon and episodes search, there are common filters such as:

- Order of the search results by relevance;
- Order of the search results by name (asc/desc);
- Order of the search results by pokedex entry (asc/desc);

There are also some filters that are specific to each type of search:

- For Pokémon queries:
 - Pokémon type;
- For episode queries:
 - Date range (start/end) that can be set for Japanese broadcast date or American broadcast date with a switch;

The website has a main content page for each of the individual entries as shown in the annexes, and it supports "more like this", that is shown in the same interface as search results page.

20 Conclusions and Future Work

At the end of the second milestone, we had prepared the first iteration of a search system that allowed us to search our collection, based on the data pipeline we had previously devised. With the work added in the final milestone, we have finished the development of the search system, exposing an improved version of it through a Search User Interface that lets users intuitively explore our collection.

In this final milestone, we have experimented with some possible improvements for our search system, mostly through an in-depth exploration of semantic search and multi-stage retrieval. While Apache Solr is still lacking in some aspects with regards to its implementation of dense vector search, we were able to work around its limitations to successfully use chunked embeddings. These limitations extend to the use of nested documents, which, along with time constraints, didn't allow us to further explore the use of our child documents to enrich results.

However, our system does not cover all use cases, and there is room for improvement. When using complex queries, closer to natural language, our systems that involve at least one lexical retrieval stage showed poor performance. With more time, query processing techniques could be explored in order to attenuate or even fix this problem.

A Appendix

A.1 Figures

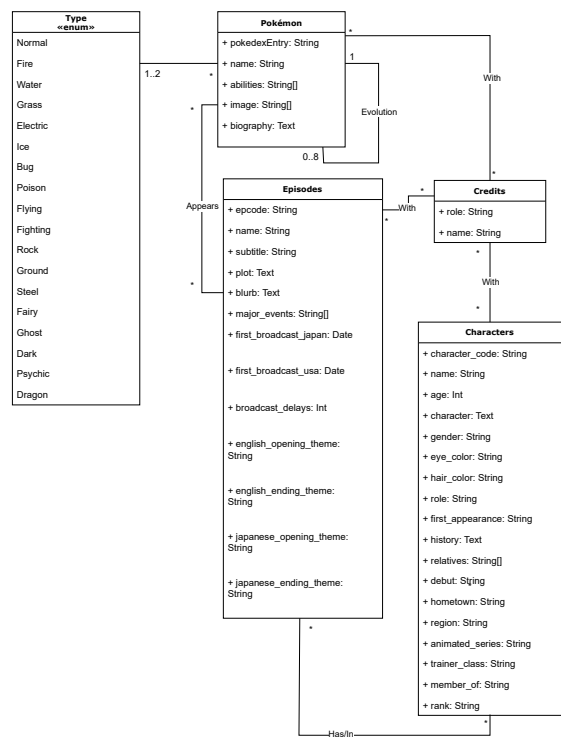


Figure 11: Conceptual Data Model

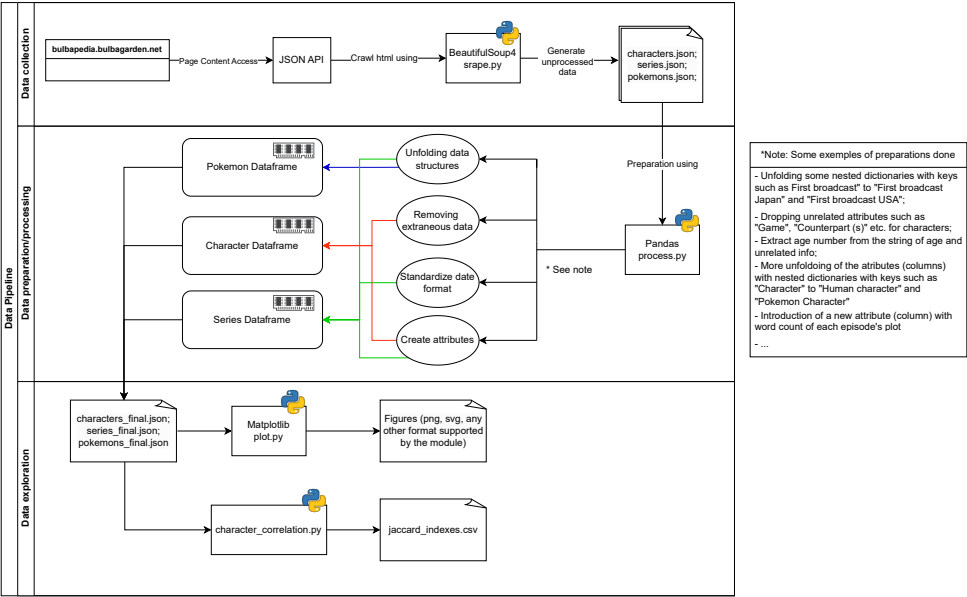


Figure 12: Data Pipeline

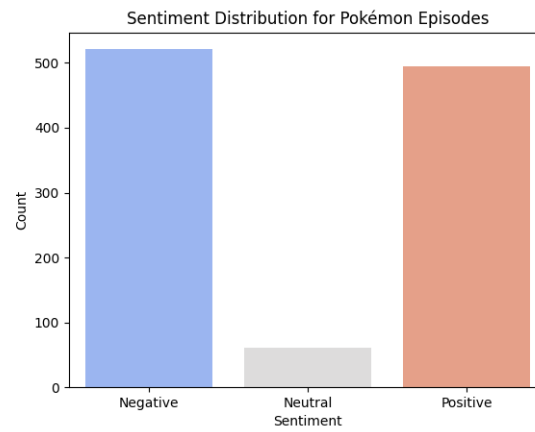


Figure 13: Sentiment Analysis

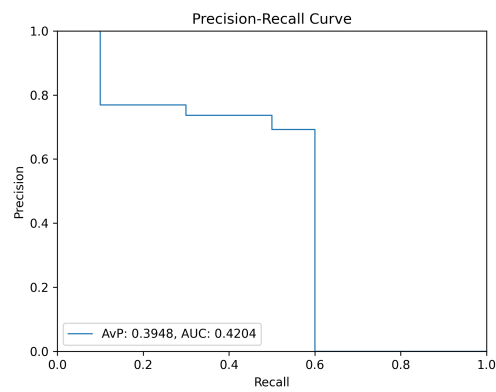


Figure 14: P-R curve for the Pokémon theft query, simple system

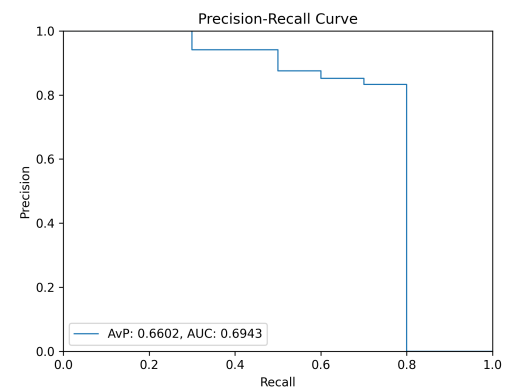


Figure 15: P-R curve for the Pokémon theft query, complex system

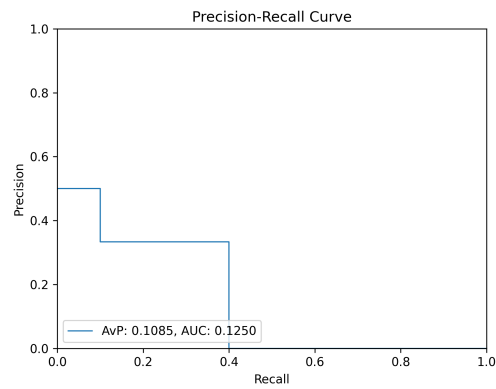


Figure 16: P-R curve for the group change query, simple system

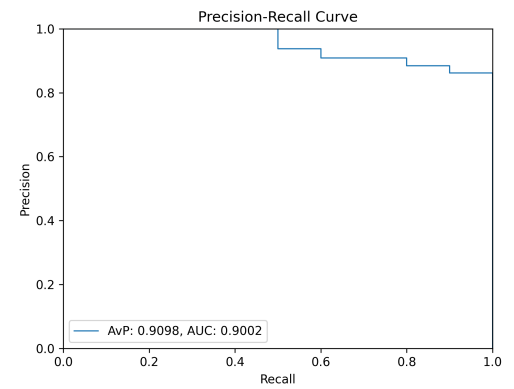


Figure 17: P-R curve for the group change query, complex system

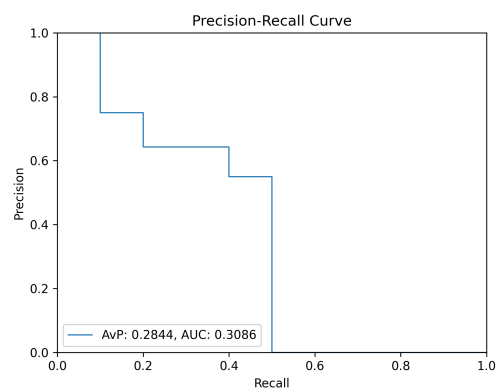


Figure 18: P-R curve for the gym battle query, simple system

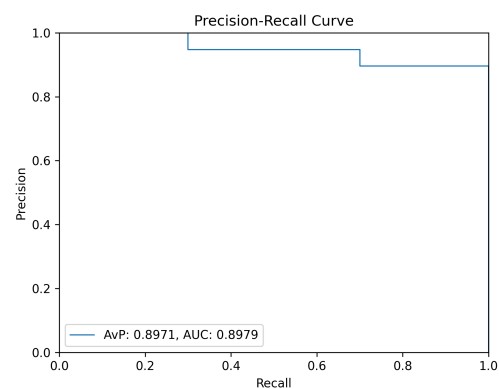


Figure 19: P-R curve for the gym battle query, complex system

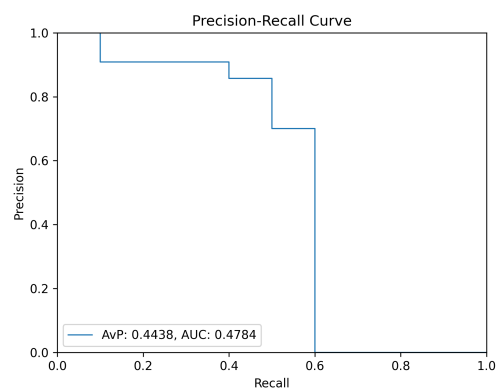


Figure 20: P-R curve for the Mega Evolution query, simple system

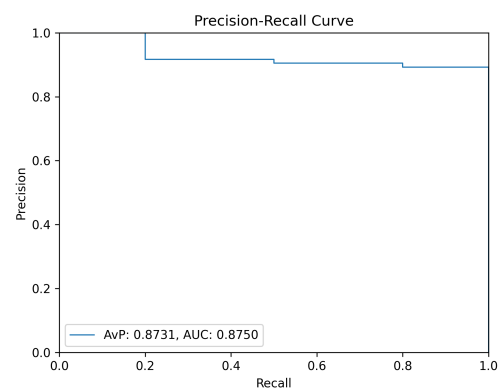


Figure 21: P-R curve for the Mega Evolution query, complex system

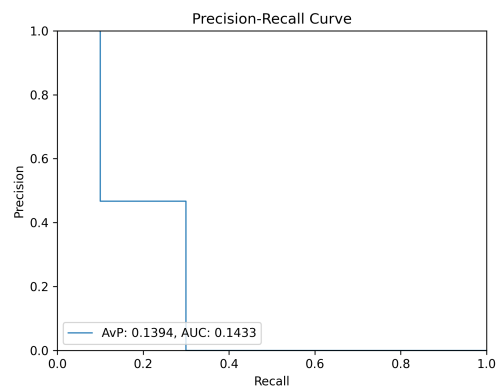


Figure 22: P-R curve for the Angry Pokémon query, simple system

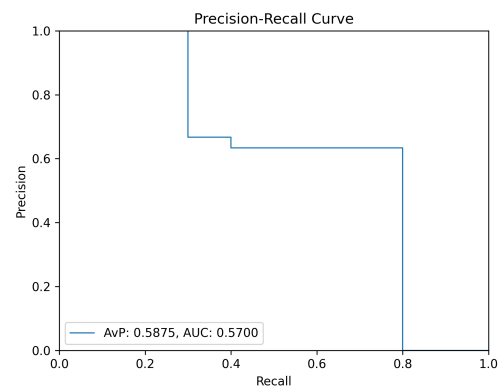


Figure 23: P-R curve for the Angry Pokémon query, complex system

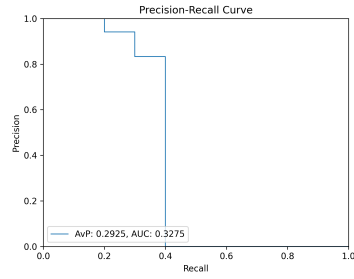


Figure 24: Revised P-R curve for the Pokémon theft query, complex system

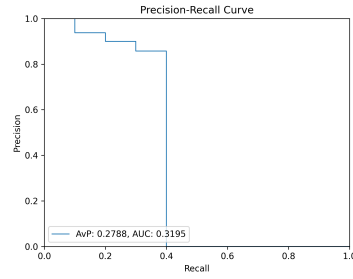


Figure 25: P-R curve for the Pokémon theft query, semantic system

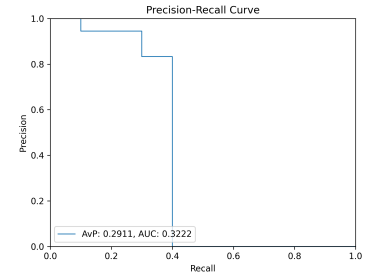


Figure 26: P-R curve for the Pokémon theft query, rerank system

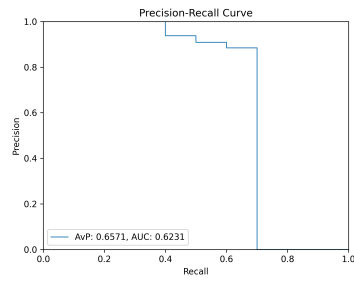


Figure 27: Revised P-R curve for the group change query, complex system

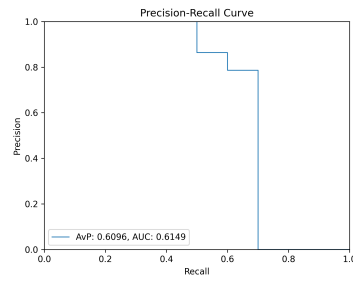


Figure 28: P-R curve for the group change query, semantic system

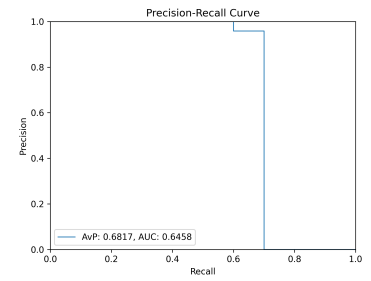


Figure 29: P-R curve for the group change query, rerank system

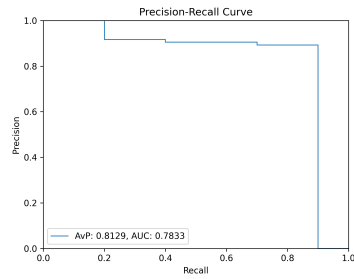


Figure 30: Revised P-R curve for the Mega Evolution query, complex system

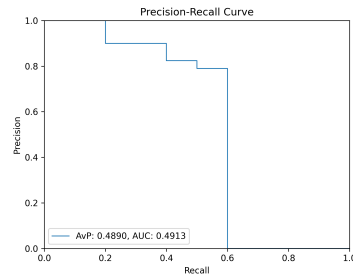


Figure 31: P-R curve for the Mega Evolution query, semantic system

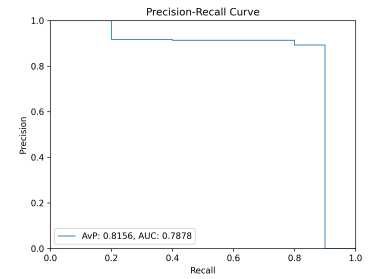


Figure 32: P-R curve for the Mega Evolution query, rerank system

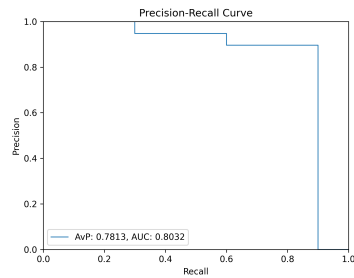


Figure 33: Revised P-R curve for the gym battle query, complex system

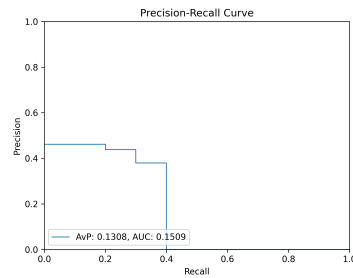


Figure 34: P-R curve for the gym battle query, semantic system

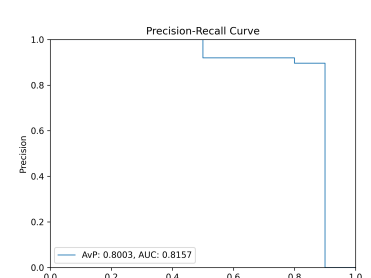


Figure 35: P-R curve for the gym battle query, rerank system

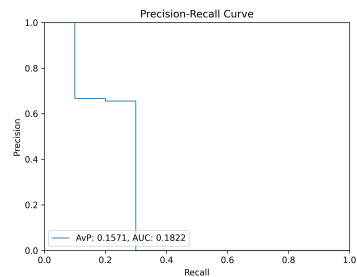


Figure 36: P-R curve for the theft sentence query, complex system

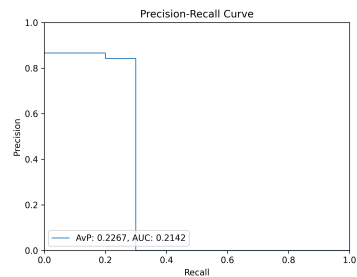


Figure 37: P-R curve for the theft sentence query, semantic system

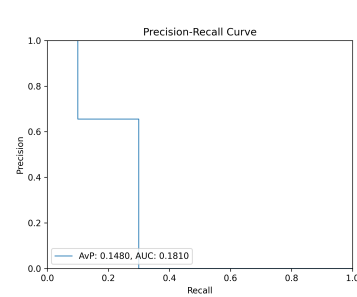


Figure 38: P-R curve for the theft sentence query, rerank system

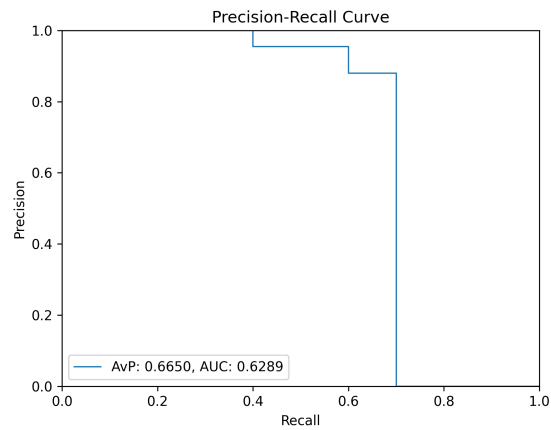


Figure 39: P-R curve for the group sentence query, semantic system



Figure 40: Welcome page of the website



Figure 41: Search page of the website

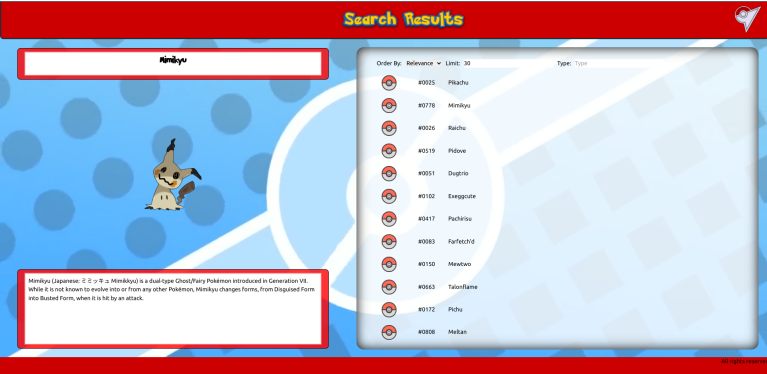


Figure 42: Search page of the website

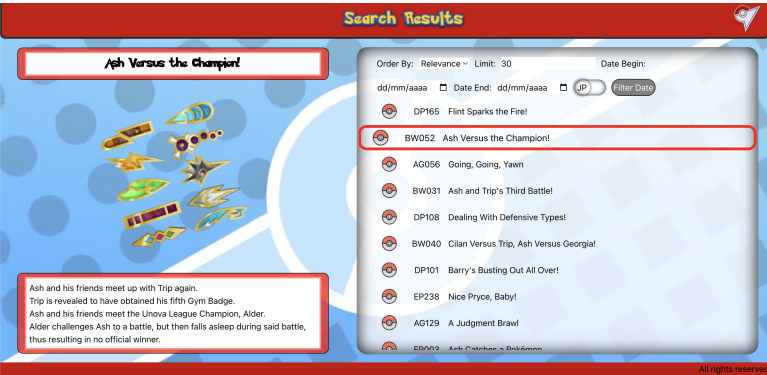


Figure 43: Episode selection of the website

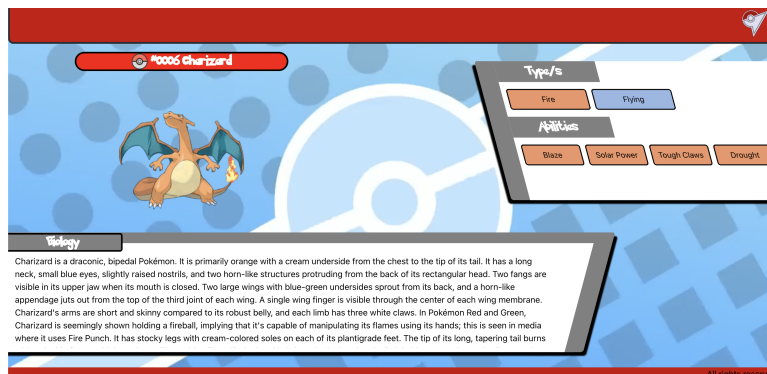


Figure 44: Pokémon page of the website

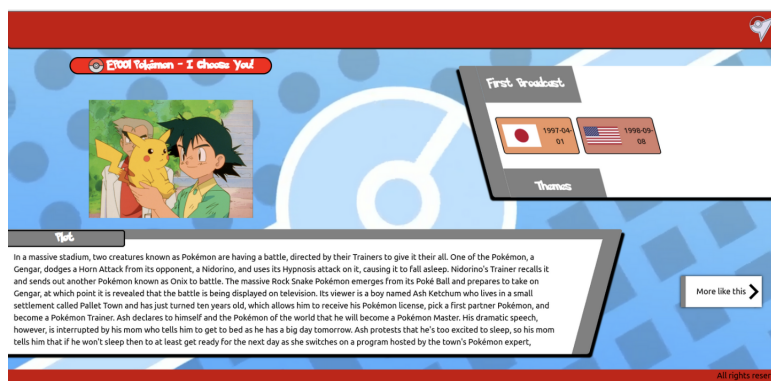


Figure 45: Episode page of the website

References

- [1] BeautifulSoup4 2024. *Beautiful Soup*. Retrieved October 10, 2024 from <https://www.crummy.com/software/BeautifulSoup/>
- [2] Bulbapedia 2024. *Bulbapedia, the community-driven Pokémon encyclopedia*. Retrieved October 10, 2024 from <https://bulbapedia.bulbagarden.net>
- [3] The Pokémon Company. 2024. *The Official Pokémon Website*. Retrieved October 10, 2024 from <https://pokemon.com>
- [4] Django 2024. *Django*. Retrieved December 16, 2024 from <https://www.djangoproject.com/>
- [5] eDisMax Query Parser 2024. *eDisMax Query Parser*. Retrieved November 17, 2024 from <https://solr.apache.org/guide/solr/latest/query-guide/edis-max-query-parser.html>
- [6] EP018 2024. *EP018*. Retrieved October 13, 2024 from <https://bulbapedia.bulbagarden.net/wiki/EP018>
- [7] EP038 2024. *EP038*. Retrieved October 13, 2024 from <https://bulbapedia.bulbagarden.net/wiki/EP038>
- [8] iso 2024. *ISO-8601*. Retrieved November 16, 2024 from https://en.wikipedia.org/wiki/ISO_8601
- [9] leaderboard 2024. *Pretrained Models*. Retrieved December 16, 2024 from https://www.sbert.net/docs/sentence_transformer/pretrained_models.html
- [10] License 2024. *Attribution-NonCommercial-ShareAlike 2.5 Generic*. Retrieved October 14, 2024 from <https://creativecommons.org/licenses/by-nc-sa/2.5/>
- [11] Matplotlib 2024. *Matplotlib — Visualization with Python*. Retrieved October 10, 2024 from <https://matplotlib.org/>
- [12] MediaWiki 2024. *MediaWiki*. Retrieved October 13, 2024 from <https://www.mediawiki.org/wiki/MediaWiki>
- [13] MLT 2024. *More Like This*. Retrieved December 16, 2024 from https://solr.apache.org/guide/8_8/morelikethis.html
- [14] Multi-value support for KnnVectorField 2024. *Multi-value support for KnnVectorField*. Retrieved December 16, 2024 from <https://github.com/apache/lucene/pull/12314>
- [15] NLTK 2024. *NLTK Python Module*. Retrieved November 17, 2024 from <https://pypi.org/project/nltk/>
- [16] Pandas 2024. *pandas - Python Data Analysis Library*. Retrieved October 10, 2024 from <https://pandas.pydata.org/>
- [17] Python 2024. *Welcome to Python.org*. Retrieved October 10, 2024 from <https://www.python.org/>
- [18] React 2024. *React*. Retrieved December 16, 2024 from <https://react.dev/>
- [19] sentence-transformers 2024. *sentence-transformers Python Module*. Retrieved December 16, 2024 from <https://pypi.org/project/sentence-transformers/>
- [20] solr 2024. *solr*. Retrieved November 11, 2024 from <https://solr.apache.org/>
- [21] solr-filters 2024. *Filters*. Retrieved November 12, 2024 from <https://solr.apache.org/guide/solr/latest/indexing-guide/filters.html>
- [22] solr-tokenizers 2024. *Tokenizers*. Retrieved November 12, 2024 from <https://solr.apache.org/guide/solr/latest/indexing-guide/tokenizers.html>
- [23] solrknn 2024. *Dense Vector Search*. Retrieved December 16, 2024 from <https://solr.apache.org/guide/solr/latest/query-guide/dense-vector-search.html#knn-query-parser>
- [24] solrparent 2024. *Block Join Parent Query Parser*. Retrieved November 17, 2024 from <https://solr.apache.org/guide/solr/latest/query-guide/block-join-query-parser.html#block-join-parent-query-parser>
- [25] solrreank 2024. *Query Re-Ranking*. Retrieved December 16, 2024 from <https://solr.apache.org/guide/solr/latest/query-guide/query-re-ranking.html#rerank-query-parser>
- [26] spacy 2024. *spaCy*. Retrieved December 16, 2024 from <https://spacy.io/>
- [27] SynonymFilter in Apache Solr 2024. *SynonymFilter in Apache Solr*. Retrieved November 17, 2024 from <https://cwiki.apache.org/confluence/display/solr/AnalyzersTokenizersTokenFilters#AnalyzersTokenizersTokenFilters-solr.SynonymFilterFactory>
- [28] WordCloud 2024. *wordcloud - PyPI*. Retrieved October 10, 2024 from <https://pypi.org/project/wordcloud/>